Assignment 2 — SecureCloud Services, Contracts and Schema

1. System capabilities

1. Register and authenticate users.
2. Manage user status and access.
3. Upload, view, list, update and delete files.
4. Create and organize folders.
5. Keep older versions of files.
6. Share a file with another user.
7. Change or revoke a file share.
8. Store file contents and retrieve them when requested.

2. Service boundaries

The design uses three services. Each service owns its data; another service accesses it only through the owner's contract. Service and team-member assignments are placeholders until the team names and roll numbers are provided.

Service	Responsible member	Data it owns
Identity Service	To be assigned: name and roll number	User accounts, password hashes, account status
File Service	To be assigned: name and roll number	Folders, file metadata, file versions and storage object keys
Sharing Service	To be assigned: name and roll number	Share records, recipient, permission and expiry

The File Service manages file metadata and calls the configured storage system for file bytes. The database stores object keys, not file contents. The Sharing Service stores user and file identifiers as references; it does not read or write another service's tables.

Calls between services

- File Service → Identity Service: `getUserAccessStatus(owner)` confirms the owner account is active.
- Sharing Service → File Service: `getFileOwner(file)` confirms the file exists and retrieves its owner.
- Sharing Service → Identity Service: `getUserAccessStatus(recipient)` confirms that the recipient account can receive a share.

The arrows are contract calls, not shared tables. A missing dependency or a denied check must produce a clear error; callers must not query another service's database directly.

See services.drawio for the editable diagram and services.png for its image.

3. Service contracts

Contracts describe behavior and public data only. They do not expose table names, SQL, or identifier formats.

Identity Service

Operation	Input	Output	Errors
`registerUser`	Name, email, password	User profile and account status	Invalid input, email already registered
`authenticateUser`	Email, password	Authentication result and access token	Invalid credentials, account disabled

`getUserAccessStatus`	User reference	Active or disabled status	User not found
`deactivateUser`	User reference, administrator authorization	Updated account status	User not found, not authorized

Passwords are never returned. The service stores a password hash, not a readable password.

File Service

Operation	Input	Output	Errors
`uploadFile`	Owner, optional folder, file name, media type, file content	File details and version	Invalid file, owner unavailable, folder not found, storage unavailable
`getFile`	File reference, requesting user	File details and download access	File not found, access denied
`getFileOwner`	File reference, authorized service caller	File owner reference and file status	File not found, caller not authorized
`listFiles`	Owner, optional folder, search text, page and page size	Matching file summaries and paging information	Owner not found, invalid filter
`moveFile`	File reference, destination folder, requesting user	Updated file details	File not found, folder not found, access denied
`deleteFile`	File reference, requesting user	Confirmation	File not found, access denied, file state conflict

Sharing Service

Operation	Input	Output	Errors
`shareFile`	Requesting user, file, recipient, permission, optional expiry	Share details	File not found, not owner, recipient unavailable, invalid permission, conflicting active share
`getShare`	Share reference, requesting user	Share details	Share not found, access denied
`revokeShare`	Share reference, requesting user	Revocation confirmation	Share not found, access denied, already revoked

4. Central operation: `shareFile`

Purpose: Give another active user a selected level of access to a file.

Input: The authenticated owner, the file to share, the recipient, the requested permission (`view` or `edit`), and an optional expiry time.

Success output: The share reference, file and recipient references, granted permission, current status, creation time, and optional expiry. It does not include file content, password data, storage keys, or database details.

Errors:

- `not_authenticated`: the caller has no valid identity.
- `file_not_found`: the file does not exist.
- `not_file_owner`: the caller cannot grant access to this file.
- `recipient_unavailable`: the recipient does not exist or is disabled.
- `invalid_permission`: the permission is not supported.
- `share_conflict`: an equivalent active share already exists.
- `dependency_unavailable`: a required service could not be reached.

Internal behavior hidden from callers:

1. Validate the requested permission and expiry.
2. Ask the File Service whether the file exists and belongs to the caller.
3. Ask the Identity Service whether the recipient is active.

4. Check for a conflicting active share.
5. Save the share in the Sharing Service's own data store.
6. Return a safe share representation; never disclose another service's tables, SQL, password hashes, or storage object key.

5. Database schema

The database sketch has three ownership groups. Foreign keys are used within a service only. User and file references in another service are opaque values, not cross-service database foreign keys. This keeps service ownership independent.

See schema.drawio for the editable ER diagram, schema.png for its image, and schema.sql for the PostgreSQL table definitions.

6. Service-properties check

Service	Reachable	Self-contained	Has a contract	Independent	Loosely coupled	Issue and fix
Identity	Yes, through its API	Yes, it owns account data	Yes, operations are listed above	Yes, account records have one owner	Yes, other services use its contract	Keep password and account tables private; do not let other services query them.
Files	Yes, through its API	Yes, it owns file metadata and versions; file bytes use its configured storage	Yes, operations are listed above	Mostly; it checks account status through Identity	Yes, that check is an API call	Use a timeout and return a dependency error if Identity is unavailable; never read Identity's database.
Sharing	Yes, through its API	Yes, it owns share records	Yes, operations are listed above	Mostly; it needs file-owner and recipient checks	Yes, both are contract calls	Use timeouts and explicit dependency errors; do not store a local copy of another service's tables.

Implementation note

This is the design benchmark for later assignments, not a claim that these three services are already deployed. The current backend is a single Express application with a health-check route. Later work can implement these boundaries as modules in that application before considering separate deployments.